SWITCH

UPDATED PRODUCT, UX & USP MASTER DOCUMENT

Smart-home routine verification, replay and regression requirements

Document purpose	**Updated single source of truth for product scope, visual direction, USP, architecture and demo.**
Key UX update	Premium interactive house interface, larger typography, meaningful state-driven motion.
Key USP update	Explain the correction → confirmed requirement → future replay workflow concretely and honestly.
Status boundary	Prototype behavior is simulated unless a real capability is implemented and tested.

Updated 22 October 2026 • Project blueprint, not a claim that every described capability is already implemented

Document map

1. 1. Executive summary
2. 2. Product problem and user need
3. 3. USP — what is actually different?
4. 4. UX direction and visual design brief
5. 5. Page-by-page product experience
6. 6. End-to-end functional workflow
7. 7. System architecture and trust boundaries
8. 8. Requirement model and API contract
9. 9. AI/ML strategy
10. 10. Data and evaluation plan
11. 11. Security, privacy and safety boundaries
12. 12. Technology and implementation sequencing
13. 13. Team responsibilities and integration
14. 14. Demo script and judging narrative
15. 15. Risks, open questions and definition of done
16. 16. Research basis and links

1. Executive summary

SWITCH is a proposed smart-home routine verification layer. It helps a household preserve explicitly confirmed corrections as reusable, versioned regression requirements, then replay those requirements when a related automation changes.

One-sentence product pitch

> **A fix today should not become a failure tomorrow. SWITCH turns a household's confirmed smart-home corrections into repeatable checks, then shows whether a proposed routine change still satisfies them before a person approves it.**

The core product is not just a smart-home dashboard, a generic chatbot, or a one-time conflict detector. The intended workflow links a household correction to a structured requirement, stores that requirement as a versioned test, and reruns it against future routine edits.

Current implementation status — be explicit

- An interactive browser prototype exists for the correction-to-regression-test concept.
- The current prototype simulates devices, routine behavior, saved lessons, test results and suggested fixes.
- The prototype is not connected to physical devices, an Amazon account, or live Alexa services; it has no configured backend, database, or live LLM.
- Any on-screen metrics, test outcomes, device states and fixes in the prototype are demonstration data, not measured performance.

This document updates the project blueprint with a stronger product interface, a clearer second-page USP explanation, and a concrete demo story. It preserves the research boundary: the differentiation is a hypothesis to validate, not a proven world-first claim.

2. Product problem and user need

The problem

Smart-home routines are often created independently, but they can interact through shared devices, occupancy, time, modes and event ordering. A routine may appear correct by itself and still cause a household outcome the user explicitly asked to avoid. After a person fixes that outcome, a later edit can accidentally reintroduce it.

Illustrative scenario

17. At night, a Goodnight routine turns off lights.
18. An occupancy condition is accidentally removed from the hallway-light action.
19. Someone is still in the hallway, but the changed routine turns the light off.
20. The household corrects the routine. Weeks later, another edit reintroduces the same failure.
21. SWITCH's intended value is to preserve the confirmed correction as a test and replay it when the routine changes again.

Why the correction must be confirmed

A single manual override does not always express a permanent rule. A person may turn a light back on because of a temporary situation, a guest, a sensor issue, or a changed preference. SWITCH should capture context, propose a structured interpretation, and ask the user to confirm whether it should become a durable requirement. Unconfirmed observations must not silently become household policy.

Product outcome

Before a proposed routine change is approved, the user can inspect the relevant saved requirements, replay result, event trace, failed assertion and proposed change. The product helps the user make an informed decision; it does not autonomously guarantee safety or deploy changes without approval.

3. USP — what is actually different?

A fix today should not become a failure tomorrow.

The differentiating hypothesis

SWITCH's proposed differentiator is a consumer-facing correction-to-regression workflow: an explicitly confirmed household correction becomes a contextual, reviewable, versioned requirement that can be replayed against later routine edits.

What that means in plain English

Most people do not want to write test scripts for their home. They notice a bad outcome and correct it. SWITCH is designed to let them explain the correction in ordinary language, confirm exactly what should remain true, and reuse that requirement as a test in the future. When a routine changes, the system does not only ask whether the new routine works in one isolated example; it also checks whether the household's previously confirmed expectation still holds in the scenarios represented by the test suite.

Concrete walkthrough

22. CORRECT — The user says: "Don't turn off the hallway light while someone is there."
23. CONFIRM — SWITCH proposes a structured requirement: if hallway occupancy is true, a Goodnight routine must not set the hallway light to off. The user confirms the scope and conditions.
24. SAVE — The requirement is stored with an ID, version, source correction, conditions, expected behavior and status. It becomes part of the household's regression suite.
25. CHANGE — A later edit removes the occupancy condition from the Goodnight routine.
26. REPLAY — SWITCH executes a deterministic modeled scenario with occupancy=true and records the routine's actions and state transitions.

27. REPORT — The requirement fails. The UI points to the action that turned off the light and shows the violated assertion.
28. REPAIR AND RECHECK — SWITCH may suggest restoring the occupancy guard. The proposed change is not accepted as correct merely because an AI suggested it; the deterministic suite must be rerun.
29. APPROVE — The person reviews the evidence and chooses whether to accept the change. A passing modeled test is evidence about the model, not a guarantee about the physical home.

How to explain the difference without overclaiming

Generic capability / claim	SWITCH's proposed focus
Detect a conflict once	Preserve a confirmed correction as a reusable requirement.
Show a suggested fix	Show which saved expectation failed, the event trace, the proposed change and the rerun result.
Use AI to explain or generate a change	Use AI only for interpretation or suggestions; deterministic checks own pass/fail.
Test a current scenario	Replay relevant saved requirements after a related routine edit.
Promise that a home is safe	Report exactly which modeled requirements passed, failed or could not be evaluated.

What we must not claim

- Do not claim "world's first," "guaranteed safe," or "prevents all conflicts."
- Do not claim that no existing product implements this workflow; the preliminary audit is not exhaustive.
- Do not present natural-language testing, conflict detection, formal verification, repair suggestions, or regression testing alone as novel.
- Do not claim a live Alexa+ integration unless it has been implemented and tested with supported access.
- Do not say that an AI-generated fix is verified until the deterministic verifier has rerun the relevant requirements.

How to validate the USP

- Compare the correction capture, confirmation, persistence and cross-edit replay workflow with Alexa Automated Testing Service, TAPFixer, SHACR, Hirz, and relevant Home Assistant / SmartThings / openHAB / Node-RED workflows.
- Interview 5–10 smart-home users about repeated routine failures, current workarounds, and whether preserving a correction as a test is valuable.
- Measure how often users confirm or edit a proposed requirement, whether saved requirements catch deliberately reintroduced regressions, and whether the event trace helps users understand the result.
- Phrase the claim as a product hypothesis until competitor review and user validation support a stronger conclusion.

4. UX direction and visual design brief

The interface should feel like a premium, interactive smart-home control product with an engineering verification layer—not a generic admin dashboard. Use the supplied reference image as the visual bar for depth, spatial layout and live device feedback. Keep the interface legible at a glance and prioritize meaningful interaction over decoration.

Visual language

- Palette: graphite and deep slate surfaces, restrained blue/cyan highlights, warm neutral text, and semantic status colors. Avoid pure-black walls of UI; use layered slate surfaces and clear contrast.
- Typography: increase the base size and hierarchy. Suggested desktop body 16–18 px, key labels 14–16 px, section headings 22–28 px, primary page title 34–44 px, and hero statement 40–56 px where space permits. Do not shrink essential information into tiny chips.
- Layout: generous spacing, clear grouping, strong alignment, consistent card radii and subtle borders. Avoid a cluttered grid of same-weight cards.
- Motion: use short, purposeful transitions for replay steps, device state changes, timeline events and pass/fail states. Respect reduced-motion preferences.

- Accessibility: keyboard navigation, visible focus, adequate contrast, semantic controls, labels for icon-only buttons, and non-color status indicators.
- Responsive behavior: desktop-first spatial view can collapse to a simplified house overview plus stacked panels on narrower screens.

Primary application shell

30. Left navigation: Home Overview, Routines, Replay Lab, Saved Requirements, Reports, Devices / Integrations, Settings. Keep the active section unmistakable.
31. Top bar: household selector, environment badge (SIMULATED / CONNECTED), search or command entry, notifications and user/profile controls.
32. Main canvas: interactive house cutaway or room map. Each room and device should be selectable; state overlays must reflect the same simulated state as the timeline and side panels.
33. Right rail: selected room/device controls, active routines, current alerts, and a compact explanation of the latest verification state.
34. Bottom action strip: Run Replay, Compare Changes, View Trace, and Review Proposed Fix. Actions should be context-aware and disabled when prerequisites are missing.

Interactive house visualization

The house view is the visual anchor, not a static hero image. Rooms and devices should respond to clicks, focus and state changes. Selecting the hallway light should reveal its current modeled state, controlling routine, relevant saved requirements and recent events. During replay, the selected device should visibly change only as the deterministic simulation applies events.

- Show device state using an icon plus text/status—not color alone.
- Allow a user to select a room or device to filter the timeline and related requirements.
- Highlight the device involved in a failed assertion and draw a restrained connector to the event or requirement details.
- Include a clear legend and avoid implying that the visual model is a live digital twin when it is simulated.

- Do not make 3D polish a substitute for functional correctness; if a real-time 3D house is not feasible, use a crisp interactive SVG/2D floor plan with the same interaction model.

Typography and content rules

- One primary headline per screen. Use concise labels but do not make critical explanation vague.
- Keep status labels explicit: PASS, FAIL, INCONCLUSIVE, SIMULATED, NEEDS REVIEW.
- Use short explanatory sentences near the result; provide deeper event details in an expandable trace.
- Never rely on a tiny tooltip to explain the USP or why a test failed.
- Keep the same type scale and component rhythm across the overview, replay lab and USP/why SWITCH page.

5. Page-by-page product experience

Page 1 — Home overview

Purpose: show the modeled home, current device/routine state, and the most relevant verification status. This is the immersive entry point, not the place to bury the USP in metrics.

- Large title: “Your home, with changes you can verify.”
- House visualization with selectable rooms and devices.
- Right rail with active routines, saved requirements at risk, latest replay status and recent activity.
- Primary CTA: “Open Replay Lab” or “Run regression check.”
- Persistent environment label: “SIMULATED DEMO” until real integration exists.

Page 2 — Why SWITCH / USP

This page must explain the difference with a concrete example, not vague phrases such as “AI-powered safety” or “smarter automation.” It should be a visual walkthrough of one correction becoming a future regression test.

Headline: “A fix today should not become a failure tomorrow.”

35. Household correction — show the exact user statement and context.
36. Confirmed requirement — display the structured rule and its version, with the user-confirmed status.
37. Routine change — show a before/after diff that removes the occupancy condition.
38. Replay — animate a short event trace: occupancy=true → Goodnight fires → hallway light set to off.
39. Failure evidence — identify the failed assertion and the exact event that violated it.
40. Proposed repair — show the guard being restored and explain why it addresses this failure.
41. Regression rerun — show the original requirement and any other saved requirements being tested again.
42. Human decision — the user reviews evidence and approves or rejects the proposed change.

Suggested supporting copy: “SWITCH is designed to remember the corrections your household explicitly confirms. When a related routine changes, it replays those requirements against the new behavior and shows the evidence. That is the product hypothesis—not a claim that conflict detection or regression testing itself is new.”

Page 3 — Replay Lab

Purpose: make the verification engine understandable and inspectable. Use a three-column or split view: routine change, event trace, and assertion results. Provide controls for Run Replay, Apply Suggested Fix, Re-run Suite and Reset Scenario.

- Show a deterministic simulated clock and ordered events.
- Show condition evaluation and state transitions.
- Display each assertion as PASS, FAIL or INCONCLUSIVE, with a reason.
- Show the routine diff separately from the AI’s plain-language explanation.
- Allow the user to inspect which saved requirement triggered the failure.

Page 4 — Saved requirements

Purpose: make the household's durable expectations inspectable and editable. Every requirement should show its source correction, scope, conditions, expected behavior, version, confirmation state and last replay result.

- Users can edit, disable or retire requirements; edits create a new version and preserve history.
- Unconfirmed interpretations remain drafts and do not count as enforced requirements.
- Show stale or untestable requirements explicitly rather than silently skipping them.
- Offer filters by room, device, routine, status and last result.

Page 5 — Reports

Purpose: provide reproducible evidence rather than a vague "safe" score. Include run ID, scenario version, requirement versions, routine version, simulator version, event trace, assertions, proposed diff and reviewer decision.

- Report test counts with PASS / FAIL / INCONCLUSIVE totals.
- Show a timeline with timestamps or simulated ticks and state changes.
- Export a report only if implemented; label demo reports as simulated.
- Never turn a passing result into a blanket home-safety guarantee.

6. End-to-end functional workflow

43. Capture correction: user describes an undesired outcome and relevant context.
44. Interpret: parser or model proposes a structured requirement with entities, conditions, expected state, scope and confidence.
45. Clarify: ask the user about missing or ambiguous conditions.
46. Confirm: user approves the meaning; only then can it become an active requirement.
47. Version: save the confirmed requirement with source, owner, version, timestamps and status.
48. Edit: a user proposes or imports a routine change into a candidate version.

49. Select tests: identify requirements and scenarios affected by the changed routine, plus relevant baseline tests.
50. Replay: execute the candidate against a deterministic simulated state and event sequence.
51. Verify: evaluate explicit assertions and produce PASS / FAIL / INCONCLUSIVE.
52. Explain: show the event trace, violated requirement, relevant condition checks and candidate diff.
53. Suggest: optionally propose a constrained repair. Keep the suggestion separate from verification.
54. Re-run: execute the same tests plus relevant regression requirements against the repaired candidate.
55. Review: user approves, rejects or revises the candidate. No silent deployment.
56. Record: retain the result, versions, trace and human decision for reproducibility.

Result semantics

Status	Meaning	Allowed conclusion
PASS	All required assertions for this run evaluated true in the modeled scenario.	The tested modeled requirements passed for these versions and scenarios.
FAIL	At least one evaluated assertion was violated.	The report must identify the requirement and event evidence.
INCONCLUSIVE	A required condition, device capability, sensor state or model behavior could not be evaluated reliably.	Do not treat as pass; explain the missing evidence.

A missing or unsupported device, unknown state, parser uncertainty, or simulator limitation must not silently turn into PASS. Preserve enough run metadata to reproduce the result.

7. System architecture and trust boundaries

Recommended logical flow: UI → API → requirement service / routine parser → versioned requirement store → deterministic simulator → assertion evaluator →

report service. An optional language model sits beside the deterministic core and proposes structured interpretations or repairs. It does not own the pass/fail decision.

Core components

- Frontend: application shell, house/room visualization, routines editor, requirement library, replay timeline, diff viewer and report view.
- API layer: authentication/household scope, schema validation, version checks, replay job creation and report retrieval.
- Requirement service: draft creation, user confirmation, version history, status transitions and traceability to source correction.
- Routine model: normalized triggers, conditions, actions, targets, priority and version.
- Deterministic simulator: simulated clock, event queue, shared device state, explicit event ordering and recorded transitions.
- Assertion evaluator: executes requirements against traces and returns PASS / FAIL / INCONCLUSIVE.
- Suggestion service: optional LLM or rule-based proposal; schema-validated and sandboxed.
- Report service: stores run configuration, requirement versions, routine versions, assertions, trace and reviewer decision.

Non-negotiable trust boundary

AI can interpret a correction or propose a repair. Only the deterministic verifier can return the test result. Only an authorized human can approve a candidate for deployment.

Replay reproducibility

- Use a fixed scenario seed or explicit event list; do not rely on wall-clock timing for test logic.
- Record routine version, requirement versions, simulator version and scenario input.
- Define event ordering and tie-break behavior in the implementation contract.

- Use identical inputs for before/after comparisons.
- Do not use model-generated text as the assertion engine.

8. Requirement model and API contract

Minimum requirement fields

Field	Purpose
requirement_id	Stable unique identifier.
household_id	Tenant / household boundary.
title	Readable requirement name.
source_correction	User's original correction, retained with appropriate privacy controls.
scope	Relevant room, device, routine or household-wide context.
preconditions	Conditions under which the requirement applies.
expected_behavior	Explicit state/action that must or must not occur.
version	Monotonic version for edits.
status	DRAFT, CONFIRMED, DISABLED or RETIRED.
confirmation	Who confirmed the requirement and when.
last_result	PASS, FAIL or INCONCLUSIVE, plus run reference.

Suggested endpoint shapes

Method / route	Purpose
POST /requirements/drafts	Create a draft from a correction; never auto-confirm.
POST /requirements/{id}/confirm	Confirm the reviewed structured requirement.
GET /requirements	List requirements by household and filter.
PATCH /requirements/{id}	Edit metadata or create a new version; preserve history.
POST /replays	Create a replay for a candidate routine

	version and selected scenarios.
GET /replays/{run_id}	Retrieve run status, assertions and trace.
POST /candidates/{id}/suggest-fix	Request a candidate repair; no deployment.
POST /candidates/{id}/approve	Record authorized human approval; deployment remains a separate, explicit capability.

These are proposed contracts, not a claim that these endpoints are already implemented. Freeze request/response schemas, status enums, error format and version semantics before parallel development.

9. AI/ML strategy

Use the smallest AI capability that improves usability without weakening verification. The project does not need to train a custom deep-learning model to demonstrate its core idea.

Appropriate AI tasks

- Convert a natural-language correction into a draft structured requirement.
- Ask clarifying questions when room, device, condition, scope or expected behavior is ambiguous.
- Summarize why a deterministic assertion failed using the trace as evidence.
- Suggest a constrained candidate edit, with a machine-readable diff and explanation.

Tasks that must remain deterministic

- Event ordering and state transitions in the modeled scenario.
- Evaluation of explicit requirement assertions.
- PASS / FAIL / INCONCLUSIVE assignment.
- Comparison of routine versions and reproducibility metadata.
- Permission checks, confirmation state and approval state.

Validation rules for model output

- Validate structured output against a schema; reject malformed fields and unknown devices/actions.
- Require user confirmation before a draft becomes an active requirement.
- Constrain proposed edits to known routine fields and show the exact diff.
- Re-run the regression suite after a proposed fix; do not infer success from the explanation.
- Provide a deterministic fallback if the model is unavailable.

10. Data and evaluation plan

Start with a small, hand-authored benchmark of representative routines and failure modes. Synthetic scenarios are acceptable for the prototype when clearly labelled; they do not establish real-world effectiveness.

Initial scenario families

- Contradictory writes to the same device.
- Occupancy / presence condition accidentally removed.
- Time-of-day or mode condition changed.
- Event ordering changes the final state.
- One routine fix breaks another previously passing requirement.
- Unknown sensor state or unsupported device capability.
- Ambiguous correction that should remain a draft until clarified.

Evaluation metrics

Metric	How to measure
Regression catch rate	Share of deliberately introduced regressions that cause the correct saved assertion to fail.
False-pass rate	Share of scenarios that violate a requirement but are incorrectly reported as PASS; prioritize minimizing this.
False-fail rate	Share of scenarios that satisfy requirements but are incorrectly marked FAIL.
Replay reproducibility	Whether identical scenario and version inputs yield identical results.

Correction-to-requirement accuracy	Human-reviewed correctness of the structured requirement generated from user language.
User comprehension	Whether participants can identify why a test failed from the trace and report.
Repair verification	Whether a suggested change passes the targeted requirement without breaking the existing regression suite.

Do not invent performance percentages or report accuracy unless they are measured on a documented benchmark. For the demo, use labelled scenario fixtures and report exactly what was tested.

11. Security, privacy and safety boundaries

- Scope every requirement, routine, trace and report to the correct household identity.
- Minimize stored raw household conversation and device data; define retention and deletion behavior before live use.
- Do not send sensitive household details to a third-party model without appropriate disclosure and consent.
- Validate all imported routine definitions and constrain model-generated changes.
- Keep candidate edits separate from live routines; no silent deployment.
- Log approvals and version changes without exposing unnecessary personal information.
- Treat unknown device state as INCONCLUSIVE, not safe.
- State prominently that simulation only validates modeled scenarios and cannot guarantee physical-world safety.

Safety statement for the product

A PASS means the selected modeled requirements passed for the recorded scenario and versions. It does not certify the real home or cover unmodeled sensors, outages, latency, device firmware, or household circumstances.

12. Technology and implementation sequencing

Pragmatic stack

- Frontend: React / Next.js or the existing prototype's current stack; TypeScript where practical; accessible component primitives and a consistent design-token system.
- Backend: Python FastAPI or a Node API—choose one for the MVP and avoid duplicate API layers.
- Persistence: PostgreSQL for households, requirements, versions, routines, replay runs and reports; local demo state is acceptable only when clearly labelled.
- Replay core: a pure deterministic module with unit tests, independent from UI and LLM calls.
- AI integration: provider adapter with structured output validation and a deterministic fallback.
- Testing: unit tests for event ordering and assertions; integration tests for requirement confirmation and replay; end-to-end test for the full demo path.

Build order

57. Freeze the data model, requirement semantics, result statuses and event ordering.
58. Make the browser demo's correction → confirm → fail → repair → rerun story reliable.
59. Implement deterministic simulator and assertion tests before connecting an LLM.
60. Add persistence and API contracts; keep draft and confirmed states separate.
61. Build the replay timeline and trace around real simulator output, not hand-coded result text.
62. Upgrade the house visualization only after state interactions and event evidence are coherent.
63. Add optional AI interpretation and constrained repair suggestions.
64. Run a deliberate regression benchmark, record results and refine the claims.

65. Add a real device/platform integration only after access, permissions and test environment are confirmed.

13. Team responsibilities and integration

Workstream	Ownership	Deliverables
A — UX / Frontend	App shell, house/room view, requirements screens, replay timeline, responsive behavior, accessibility.	Design tokens, component contracts, interactive pages, end-to-end UI flow.
B — Deterministic core / Backend	Routine schema, simulator, event ordering, assertion engine, API and persistence.	Unit-tested replay engine, API schemas, migrations, reproducible run reports.
C — AI / Evaluation / Integration	Correction parser, clarification flow, constrained repair proposals, benchmark and report interpretation.	Validated structured output, prompt/schema tests, evaluation scenarios, integration tests.

The team may reassign these roles, but each deliverable must have one accountable owner. Agree on a single shared contract for routine objects, requirement objects, replay inputs, result statuses and trace events.

Minimum trace event

Each event should include a sequence index or simulated timestamp, routine ID/version, action, target device, prior state, resulting state, condition evaluations and source event. Assertions should reference the relevant trace event IDs.

14. Demo script and judging narrative

A 90–120 second demo

66. Open Home Overview. Explicitly show the SIMULATED DEMO label.
67. Select the hallway light and introduce the saved requirement: “Do not turn off the hallway light while occupancy is true.”
68. Open the routine diff and show that the occupancy guard was removed.

69. Click Run Replay. Animate occupancy=true, Goodnight routine fired, light changed to off.
70. Show FAIL with the precise assertion and event trace—not just a red badge.
71. Click Apply Suggested Fix. Show the proposed diff restoring the occupancy guard.
72. Click Re-run Regression Suite. Show the same scenario now passing in the simulator.
73. Show the saved requirement's version and run report, then explain that the user must approve the candidate.
74. Close with the honest USP: "We are testing the hypothesis that turning confirmed household corrections into reusable regression requirements makes routine changes easier to verify over time."

Judge questions to prepare for

- What is different from Alexa Automated Testing Service? Answer with the consumer-facing correction capture, confirmation, versioned requirement and replay workflow as the hypothesis—not a categorical claim that ATS cannot do any similar thing.
- Is it actually connected to Alexa? State the current prototype status exactly. Do not imply integration that is not implemented.
- How do you know the fix is safe? You do not guarantee real-world safety; you show deterministic evidence for modeled requirements and scenarios.
- Why use AI? To interpret human language and suggest constrained edits, not to decide test outcomes.
- What happens if the correction is ambiguous? Ask a clarifying question and keep it as a draft until confirmed.
- What proves the USP? A comparative prior-art review plus user validation and benchmark evidence; the current audit is preliminary.

15. Risks, open questions and definition of done

Risk	Mitigation
USP overlaps with existing tools	Continue prior-art audit; make claims

	narrower and evidence-based.
Pretty UI hides a weak core	Drive the timeline and report from deterministic replay output.
AI misinterprets a correction	Schema validation, clarification, user confirmation and fallback.
False PASS due to missing data	Use INCONCLUSIVE for unknown or unsupported conditions.
Nondeterministic event order	Define tie-break rules and test them.
Prototype appears live	Persistent SIMULATED DEMO badge and explicit copy.
Fix repairs one test but breaks another	Re-run the relevant saved regression suite.
Scope expands into a full smart-home platform	Keep the MVP centered on correction capture, replay, evidence and approval.

Definition of done for the MVP

- A user can create a correction draft, review its structured interpretation and explicitly confirm it.
- Confirmed requirements have stable IDs and version history.
- A changed routine can be replayed against saved requirements in a deterministic model.
- The replay report includes inputs, versions, ordered trace events and assertion results.
- PASS, FAIL and INCONCLUSIVE have documented semantics.
- A suggested repair is displayed as a diff and is not treated as verified until rerun.
- A passing repair is tested against the relevant regression suite, not just one assertion.
- The UI uses legible typography, accessible contrast, visible focus and meaningful state-driven motion.
- The interface labels simulated data and does not imply physical device control.
- The demo can be reset and replayed consistently.
- The USP language remains qualified until prior-art and user validation are completed.

Final product principle

Build a verifiable workflow first; make it beautiful without letting visual polish imply capabilities the system does not have.

16. Research basis and links

The following sources informed the preliminary competitive positioning. They do not constitute an exhaustive prior-art search or legal novelty opinion.

- Amazon Alexa Automated Testing Service — https://developer.amazon.com/docs/alexaplus/smarthome/automated-testing-service.html
- Amazon ATS: Create test plans — https://developer.amazon.com/docs/alexaplus/smarthome/ats-create-test-plans
- TAPFixer research — https://arxiv.org/abs/2407.09095
- SHACR research — https://arxiv.org/abs/2606.22312
- Hirz repository — https://github.com/BashaarJavaid/Hirz
- User-centric evaluation of smart-home conflict resolution — https://experts.illinois.edu/en/publications/a-user-centric-evaluation-of-smart-home-resolution-approaches-for/
- Alexa+ add-ons — https://developer.amazon.com/docs/alexaplus/add-ons/home
- Alexa+ MCP toolkit quickstart — https://developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart

Research conclusion

The preliminary review indicates that generic conflict detection, repair, natural-language test plans and regression testing are not defensible as unique by themselves. The proposed angle to validate is the consumer-side process of turning explicitly confirmed household corrections into contextual, versioned, reusable regression requirements across future edits. More research and user testing are required before claiming that this workflow is unique.